Credible Link Flooding Attack Detection and Mitigation: A Blockchain-Based Approach

Xiaofeng Jiang, *Member, IEEE*, Qianbao Shi, Hengkun Miao, Wanqin Cao, Huasen He, *Member, IEEE*, Shuangwu Chen, *Member, IEEE*, and Jian Yang, *Senior Member, IEEE*

***Abstract*—Due to the concentrated distribution of network traffic, the Internet is highly vulnerable to link flooding attack in Distributed Denial-of-Service attacks (DDoS-LFA), which utilizes the legitimate low-rate attack traffic to block the selected network area. In recent years, building trusted networks has been considered as a promising strategy to address the security issues. Nevertheless, deploying a trusted link defense mechanism in the attacked network area faces many challenges imposed by the smart scheme and legitimate disguise of DDoS-LFA. In order to overcome these challenges, we propose a blockchain-based DDoS-LFA detection and mitigation scheme, named CREDIT, to guarantee the security of attacked area, while existing works only use blockchain to share the detection results of traditional solutions. CREDIT uses blockchain to record and share the information of links and flows in real time, which enables routers in the protected area to easily trace the paths of all active flows and capture the fragile links. On the basis of link features, a credible deep learning method performed on randomly selected nodes is proposed to detect DDoS-LFA against data spoofing. When an attack alarm is raised, CREDIT performs similarity analysis to locate attackers and migrate suspicious traffic based on the flow features of alarm links. Experimental results based on real implementation and attack testbed show that, by integrating blockchain, CREDIT performs better than traditional non-blockchain-based DDoS-LFA defense methods when faced with data tampering.**

***Index Terms*—DDoS-LFA, trusted link defense mechanism, blockchain, distributed attack detection, attack sources identification.**

I. Introduction

DUE TO the simple principles, diverse schemes and significant effects [1], [2], Distributed Denial-of-Service (DDoS) attack has long been one of the attack methods that threaten the normal network communications, such as the 1.35 Tbps attack against Github in 2018 [3], and the 2.3 Tbps attack against Amazon cloud service in 2020 [4]. To avoid being defended by anomaly detection measures [5], DDoS attack has developed a Link Flooding Attack (DDoS-LFA) mechanism, which can cut off the connection between selected invasion area and the Internet by flooding specific upstream links, and hence escape the inspection of end-hosts [6]. For example, the Coremelt attack [7] can flood the backbone links and cause the network to be paralyzed. The Crossfire attack [8] can flood only a few network links to cut off network connections of a large number of publicly accessible servers. Since the flow density of the Internet follows a *power-law* distribution [9], most of the Internet traffic are concentrated on a small number of links, which will lead to serious vulnerability to DDoS-LFA [10]. For instance, the attack targeting NetEase resulted in a five-hour outage of service in 2015 [11], and the one targeting Spamhaus congested links of four major Internet exchange points in Europe and Asia in 2013 [12]. As DDoS-LFA attacks can easily cause huge impacts such as long-time out of service (especially for large corporations and important departments), proposing a trusted link defense mechanism to protect the important area in the network and mitigate the impact of the DDoS-LFA attacks is a meaningful and attractive topic. In a protected area, link information is recorded and shared among trusted nodes. The aggregation of correlated abnormal traffic can be seen as an expression of DDoS-LFA, and thus the defense mechanism can detect attacks and identify suspicious sources.

However, designing a trusted link defense mechanism against DDoS-LFA is confront with three main challenges. The first challenge is to identify the potential target links before DDoS-LFA occurs. The dynamic characteristics of DDoS-LFA make attack flows easily bypass the countermeasures deployed in fixed network locations [13]. Woodpecker proposed by [14] can locate the congested links and detect DDoS-LFA based on a judgment algorithm [15]. When the suspicious target links were identified, Aydeger et al. [16] and Lee et al. [17] provided collaborative rerouting methods between several network regions to cope with the elusiveness of target links, and mitigated link congestion. However, the rerouting mechanism introduced an additional server in each network region to maintain routing control messages, which greatly increased the deployment cost and end-to-end latency. Therefore, it is critical to identify the underlying target links before DDoS-LFA

Manuscript received 20 April 2023; revised 30 November 2023; accepted 15 January 2024. Date of publication 23 January 2024; date of current version 12 July 2024. This work was supported by the National Natural Science Foundations of China (Grant No. 62341113, 62173315, 62101525, 62201543, 62021001 and U23A20275), by the Youth Innovation Promotion Association CAS (Grant No. 2020450), by the China Postdoctoral Science Foundation (Grant No. 2023M733422), by Anhui Provincial Major Science and Technology Project (Grant No. 202103a05020024), by the Fundamental Research Funds for the Central Universities, and by the China Environment for Network Innovations (CENI). The associate editor coordinating the review of this article and approving it for publication was A. Veneris. *(Corresponding author: Huasen He.)*

The authors are with the Department of Automation, University of Science and Technology of China, Hefei 230027, China, and also with the Institute of Artificial Intelligence, Hefei Comprehensive National Science Center, Hefei 230026, China (e-mail: jxf@ustc.edu.cn; shiqianbao@iai.ustc.edu.cn; miaohengkun@mail.ustc.edu.cn; lccwq@mail.ustc.edu.cn; hehuasen@ustc.edu.cn; chensw@ustc.edu.cn; jianyang@ustc.edu.cn).

Digital Object Identifier 10.1109/TNSM.2024.3357660

occurs, so that the network can take defensive measures promptly.

The second challenge is to capture the low-rate attack flow of DDoS-LFA, which may be legitimate and does not contain malicious packets. Hirayama et al. [18] utilized the *ChangeFinder* scheme [19] to analyze the changing packet patterns of different Autonomous Systems (ASes) suffering from DDoS-LFA. Their experimental results showed that the distinguish ability declined rapidly with the growth of network scale. Kang et al. [20] performed a rate change test, which temporarily increased the bandwidth of core links and then observed the response to discern attack traffic. Nevertheless, this scheme was designed against cost-sensitive attackers, and failed to handle core links. Some other schemes used the Software Defined Networking (SDN) to capture attack flows [21], [22], while DDoS-LFA may isolate the switch policy from other data plane devices, resulting in routing inconsistencies [23].

Finally, it is challenging to identify the malicious sources, when attackers of DDoS-LFA typically utilize real IP addresses. Zargar et al. [24] proposed a route-based packet filtering scheme to identify unexpected source addresses (i.e., spoofed addresses) and drop relevant packets. However, when an attacker used genuine IP addresses, the routers were unable to filter such IP packets. To track attack flows using genuine source addresses, Zheng et al. [22] proposed a SDN-based DDoS-LFA locator for networks enabled by OpenFlow specification. The locator initially treated all addresses as suspicious source addresses, and categorized them according to their prefixes. After performing multiple rounds of prefix analysis, the number of suspicious flows can be shrank and the attacker can be finally located. However, this SDN-based scheme will be ineffective when the control channel is blocked, and the adaptive changes of target links will lead to failure of prefix analysis. The comparison of existing systems is summarized in TABLE I.

The blockchain technologies including both permissioned and permissionless blockchain [25], [26] provide a promising solution for handling DDoS-LFA, since information recorded in blockchain is non-tampering, verifiable, traceable and highly secure [27].

In order to overcome the above three challenges caused by the elusiveness of DDoS-LFA, we develop a blockchain-based DDoS-LFA detection scheme, called CREDIT, which builds blockchain into the network to establish a trusted link defense mechanism and protect the connectivity of the network. In the built-in blockchain, the consensus processes are used to monitor the fragile links and extract features to discover abnormality, which are straightforwardly utilized to identify suspicious sources and mitigate the congestion. The smart contract writes the detection and identification results into the blockchain to provide a decentralized secure, flexible and low-cost collaboration among routers in the protected area. By integrating the blockchain, CREDIT performs much better than traditional non-blockchain-based DDoS-LFA defense methods.

The CREDIT solution presents an innovative approach for mitigating DDoS-LFA attacks, surpassing the limitations of conventional methods in the realm of network security. By integrating blockchain technology into cybersecurity and utilizing it for real-time recording and sharing of information pertaining to links and traffic flows, it facilitates efficient tracking and identification of network traffic while minimizing the impact caused by DDoS-LFA. Thus, CREDIT offers a dependable, adaptable, and cost-effective mechanism for safeguarding the network. The fundamental contributions of this work are summarized as follows.

1) Blockchain-based link information collection: CREDIT leverages blockchain technology to document and disseminate real-time information about connections and data movement. This capability allows routers within a protected area to effortlessly track the routes of all currently active data streams. Hence, the suspicious target links and sources can be located by conducting correlation analysis based on the reliable and real-time record collection. To save computing and storage resources of the routers, as well as minimizing the amount of content stored in the blockchain, only a few nodes are selected to deploy CREDIT by solving a Vertex Cover Problem (VCP). Based on the real-time blockchain record, we can discover the potential target links by calculating the flow density of every link, and figure out the target links before the attack. Thus, the attack detection will be more efficient.

To the best of our knowledge, it is the first work that applies blockchain to detect sophisticated DDoS-LFA, while the existing works only use blockchain to share the blacklisted IP addresses provided by traditional detection schemes.

2) Credible intelligent DDoS-LFA detection and mitigation: The artificial intelligence and blockchain are combined to endow CREDIT with attack detection and mitigation capabilities. The blockchain collects real-time link information of the protected area to capture the attack patterns and identify suspicious sources. Traditional intelligent attack detection algorithm uses historical data and is vulnerable to false past data, the developed credible deep learning attack detection method can still predict normal data even if an attack is occurring. The trustworthiness of the historical data is evaluated via the deviation between the prediction and actual value, and then the historical data is modified according to its trustworthiness to improve the input of the detection module.

Hence, CREDIT can overcome the attack destruction of collected data, and locate attackers by identifying the hidden attack features in suspicious flows.

3) Real implementation: The CREDIT prototype is implemented on China Environment for Network Innovations (CENI) platform with Ethereum. Its performance is evaluated on a typical DDoS-LFA testbed in terms of efficiency, security and cost effectiveness. Our experimental results show that CREDIT has a significant performance raise compared with the traditional non-blockchain-based methods when facing DDoS-LFA attacks. It produces a more sensitive detection of attack links and a more accurate identification of the DDoS attackers. Furthermore, if the data of our system is tempered by mistake or maliciously, the performance of the system is still stable without any huge decline. While achieving the benefits above, the necessary resource for CREDIT's algorithm is

TABLE I
COMPARISON OF EXISTING SYSTEMS WITH CREDIT

Example system	Technique methods	Problems of the methods	Solutions of CREDIT
Woodpecker	Locating congestion and determining attack types based on global information	Difficulty in collecting information, complex processing mechanisms	Record network events on the distributed and tamper-proof nature of blockchain, and process them in real-time using deep learning to eliminate the need for waiting to collect and process global information.
Codef	Rerouting based strategies to avoid or mitigate LFA attacks	High network latency and jitter, and requires additional resources	
Spiffy	Avoiding LFA Attacks by Increasing Attacker Attack Costs	Lack of counter strategies for core link attacks	Continuous monitoring of the network's changing state through deep learning, with a particular emphasis on core connection traffic data, alongside smart contracts for automating network management and executing pre-defined strategies in response to potential attacks.
RADAR	Locating attackers through address analysis and filtering	Becomes ineffective when the control signal is obstructed or when the target link adjusts adaptively.	

slight. Our system thus provides a solution which has excellent performance for preventing and mitigating DDoS-LFA attacks at a low network resource cost. It also provides a new mind that combine blockchain with network attack detection and mitigation.

The remainder of this paper is organized as follows. In Section II, we give a brief description of blockchain and our threat model, as well as present some existing work of blockchain-based defense schemes against DDoS attack. We detail the design and key technology of CREDIT in Section III and Section IV, respectively. In Section V, we conduct experiments to evaluate the efficiency and overhead of CREDIT. Finally, we conclude the paper in Section VI.

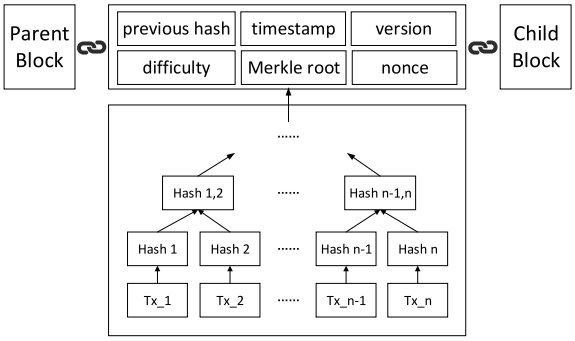


Fig. 1. Structure of blockchain.

II. PRELIMINARY

A. Blockchain

Blockchain technology was first introduced in Bitcoin [28], where it was used as a secure public ledger to achieve Bitcoin transactions without a central authority. In blockchain, the data stored by applications are confirmed by all the nodes in the network to ensure authenticity and consistency. Since blockchain is a decentralized system where each node stores complete data, an attacker is unable to tamper with the data unless it has more than 51% of the entire network's computing power [29]. Recently, through smart contract [30], the application of blockchain is no longer limited to digital cryptocurrency, but expands to other fields, such as decentralized Internet of Things (IoT) [31], smart healthcare [32], smart grid [33], smart city [34], etc.

The consistency, immutability, traceability, and security of blockchain mainly come from the chain structure. As shown in Fig. 1, blockchain combines data blocks in chronological order, and each block generally includes a block header and a block body. The header encapsulates the hash value of the parent block, timestamp, the current version number, difficulty, the Merkle root [35] and a nonce value. The body includes transactions between different entities and the Merkle tree generated by these transactions through a double hash process. In a blockchain network, all the nodes and devices are called as *miners*, which are constantly competing with each other for transaction-packaging rights through a consensus algorithm. This process is called as *mining*. The node successfully winning the right to package block is referred to as *winning miner*. When a new block is generated by the *winning miner*, other nodes will verify it and append the valid block to the previous one to update the blockchain.

Except for Bitcoin, the merits of blockchain have promoted the development of various blockchain systems, such as Ethereum [36], a platform for developing decentralized applications using smart contract; Hyperledger [37], an architecture for customizing networking rules. These systems provide us with an opportunity to develop our distributed applications for defending network attacks.

As for the consensus algorithms, PBFT [38] is a practical Byzantine fault tolerance (BFT) solution for asynchronous networks, used by Hyperledger Fabric as its consensus protocol. However, PBFT has quadratic message complexity [39], which limits its scalability under many failures. Several new BFT algorithms have been proposed to achieve high-throughput and low-latency consensus under faults and asynchrony. HotStuff [40] uses threshold signature to reduce message complexity to linear, but it suffers from frequent leader changes under failures. Prosecutor [41] penalizes suspected faulty replicas to improve consensus efficiency, avoiding performance degradation from repeated attacks by faulty servers in HotStuff. Narwhal [42] adopts a DAG-based consensus mechanism, which allows parallel processing of transactions that are sequential in blockchain. It achieves fully asynchronous consensus and significant improvement in transaction efficiency compared with Hotstuff. However, we did not employ BFT consensus algorithms because they tend to be centralized, posing a risk of single-point attacks. In contrast, Proof of Work (PoW) permissionless blockchains are decentralized, comprising a network of numerous nodes, each holding an identical copy of the data. This structure ensures that if one node is compromised, the others can still function

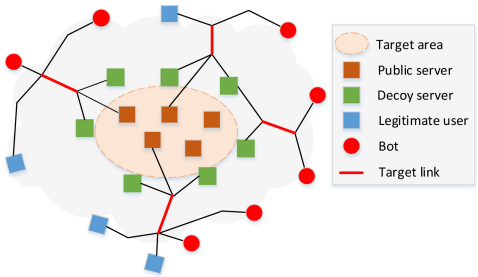


Fig. 2. Pattern of the Crossfire attack.

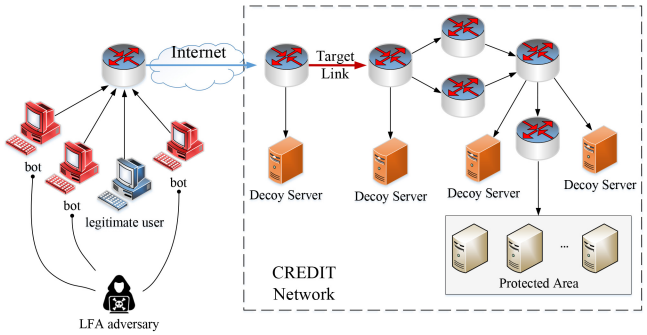


Fig. 3. Deployment strategy of CREDIT.

seamlessly. Furthermore, it's difficult for attackers to focus their efforts on a specific location or node in such a network. To significantly disrupt the network, attackers would need to target multiple nodes simultaneously, which is a complex task. Considering the advantages in terms of tamper resistance due to decentralization and distributed PoW consensus mechanisms, we select a PoW permissionless blockchain instead of a permissioned blockchain.

B. Attack Model

In contrast to traditional DDoS attack, the target of DDoS-LFA is not the selected servers, but specific links around the selected area. We choose the Crossfire attack shown in Fig. 2 as an example. It mainly contains four elements: target area, target link, public server (the victim of DDoS-LFA) and decoy server, whose flows are imitated to ensure the rates of attack flows are similar to these of legitimate flows. The attacker uses Pay-Per Install services [18] to control bots (devices controlled by the attacker and used in attack) to send a large amount of low-rate traffic to decoy servers through the target links. As a result, the target links are congested and the target area is completely disconnected from the Internet.

The detailed attack procedure is as follows. Firstly, the attacker will select two sets of servers as public servers and decoy servers after determining the target area. The former are located within the target area and denoted as $Ps = \{Ps_1, Ps_2, \ldots, Ps_N\}$, while the latter are located around the target area and represented by $Ds = \{Ds_1, Ds_2, \ldots, Ds_M\}$. Next, the attacker instructs a set of bots, denoted as $B = \{B_1, B_2, \ldots, B_O\}$, to run multiple *traceroutes* to find all paths to *Ps* and *Ds*. Each path of a bot-server pair (i.e., (B_i, Ps_j) or (B_i, Ds_j)) includes a list of IP addresses assigned to the interfaces of routers on this path, where the two IP addresses of adjacent routers' interfaces identify a link. With these IP addresses and link information, the attacker then builds a link map containing all the links between adjacent routers and servers. After that, the attacker calculates the flow density and selects target links with high flow density for flooding. We denote the target links by $L = \{l_1, l_2, \ldots, l_K\}$. Finally, the attacker assigns each bot a list of decoy servers, denoted as $Ds_l \subseteq Ds$, and the packet sending rates to the servers in Ds_l are denoted as Vs_l. The bots ensure that the rates of attack flows are similar to these of legitimate flows, and the aggregated flow rates are slightly higher than the bandwidths.

Three characteristics of DDoS-LFA [8] can be observed from the above attack procedure. 1) The attacker selects links with high flow density to flood, so as to congest the vital links with the least cost. 2) The attacker controls bots to send much low-rate traffic to congest target links, leading to a sudden increase in the number of packets. 3) The attacker installs the same attack program on bots, hence the sending rates and packet size of bots are analogous. Taking these features into account, we design the CREDIT system to discover the potential target links of DDoS-LFA, detect attack with low-rate traffic, identify suspected sources and mitigate the congestion.

C. Existing Blockchain-Based Works for DDoS Defence

There have been some studies [43], [44], [45], [46], [47], [48] using blockchain to defend DDoS attack. Meng et al. [43] have provided a review of blockchain-based intrusion detection, which indicated that these two technologies can complement each other, and pointed out two challenges for data sharing and trust computation. Alexopoulos et al. [44] presented a collaborative defense mechanism, which utilized blockchain to mitigate DDoS. Spathoulas et al. [49] proposed a scheme of sharing malicious IP addresses in blockchain. Essaid et al. [50] used blockchain and deep learning methods to identify illegal traffic. The existing works were designed to detect traditional DDoS attacks, such as Synchronize Sequence Numbers (SYN) flooding and User Datagram Protocol (UDP) flooding, by analyzing the traffic arriving at the end-hosts and sharing blacklisted IP in blockchain. However, these schemes failed to tackle DDoS-LFA, which utilizes legitimate-like flows and can not be detected at the end-hosts. Therefore, it is in urgent need to propose a solution which can be easily deployed and defend DDoS-LFA.

III. Design of CREDIT

In this section, we give a system overview of CREDIT. Since the attack flows of DDoS-LFA are always aggregated in the links that are not far from the target area, it is feasible to deploy CREDIT around the protected area to gain an appropriate tradeoff between the security and efficiency. Fig. 3 shows the deployment strategy. As the number of blockchain nodes increases, the number of transactions and the required computing resources also increase. Thus, the deployment of CREDIT should balance system efficiency and resource

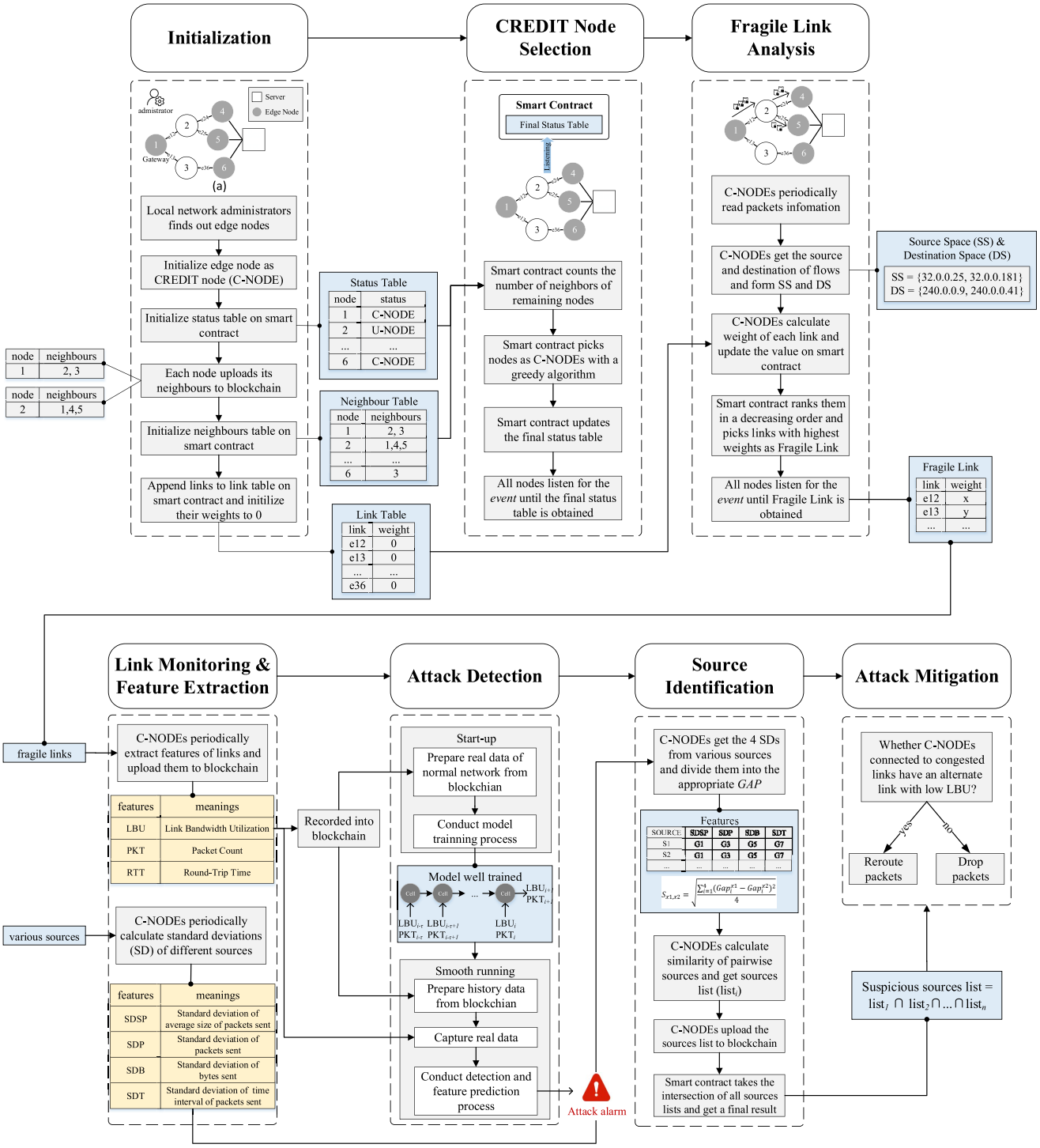


Fig. 4. System overview.

consumption. Because the protected area (or target area) lies in a local network, and the routing radius usually does not exceed ten hops, we choose the public and decoy servers around the protected area, and routers that are less than ten hops away from the area to deploy CREDIT.

The CREDIT system consists of 7 modules as shown in Fig. 4. The work flow is as follows: Firstly, the network administrator conducts initialization process to generate the status table, neighbour table and link table. Then, the blockchain uses status table and neighbour table to select CREDIT nodes, which are responsible for detecting and mitigating attacks. Next, the CREDIT nodes analyze fragile links and monitor these links to find a clue to DDoS-LFA. They periodically extract features of fragile links and various sources, and utilize these features to conduct attack detection and source identification. At last, the CREDIT nodes mitigate congestion caused by DDoS-LFA by exploiting the list of identified bots. Once the system is set and started up properly,

it can detect the attacks and protect the devices and data in the designated network area effectively. With the help of blockchain technology which can provide highly reliable data, the routers are able to trace the active flows, share real-time data of the network status between each other, analyze the data correlatedly and identify the attack links by their flow density. The characteristic of the attacks can be captured and the attacker can be precisely located from these data with the participation of artificial intelligence, and the routers can quickly restrict the flows and mitigate the impact of the attack while not intercepting normal links and data flows. Experiment results mentioned in the remainder will show that with the combination of blockchain and AI, our system is significantly more effective in attack detection and mitigation compared with the traditional algorithms. As mentioned in the former chapter of this paper, only part of the routers in the protected area need to deploy the CREDIT algorithm, and the cost of the system, including computing resource, storage and network bandwidth, are slight.

A. Initialization Module

This module is used to initialize the whole system. Firstly, the local network administrator selects the gateway router and routers connected to servers as CREDIT nodes. The CREDIT nodes locate at the edge of the local network, hence providing a coverage of the target network. Next, the network administrator initializes a status table on blockchain. The status table containing C-NODE, N-NODE and U-NODE is used to record the status of each node, where C-NODE denotes CREDIT node responsible for fragile link monitoring and information verification, N-NODE represents normal node, and U-NODE refers to the node whose state is unknown. The administrator also initializes an empty neighbour table and an empty link table on blockchain. The neighbour table is used to record neighbours of each node. The link table records weights of different links, while each weight represents the number of flows that may be transmitted through the corresponding link and is initialized to 0. Then each node of the network uploads its neighbors and the corresponding links to the neighbour table and the link table, respectively.

Take Fig. 4(a) as an example, node 1 has 2 neighbours, namely node 2 and node 3. Correspondingly, node 1 maintains 2 links, namely link e12 and link e13. Therefore, in the neighbour table the neighbours of node 1 has 2 elements, namely 2 and 3. The link table contains e12 and e13, which are initialized to 0. The status table and neighbour table will be used by the CREDIT Node Selection module to select nodes to inspect links, while the link table will be used by the Fragile Link Analysis module to obtain potential target links.

B. CREDIT Node Selection Module

This module aims to select minimun C-NODEs to monitor all links. C-NODEs upload important traffic information to blockchain and participate in verifying the information received from other C-NODEs, resulting in a great many interactions between these nodes. Therefore, lessening the number of C-NODEs can reduce network synchronization overhead and improve data verification efficiency. In addition, the selection of C-NODEs should ensure that each link connects to at least 1 C-NODEs. The blockchain first picks out the neighbours of all nodes except the gateway router and routers connected to servers. Then, a greedy algorithm detailed in Section IV-A is employed to select the C-NODEs, which are responsible for monitoring the links and extracting features to facilitate detecting and mitigating of DDoS-LFA. Then, the C-NODE selection result will be synchronized to all nodes.

C. Fragile Link Analysis Module

This module aims to discern fragile links, which have high flow density and are the most likely to be congested. In DDoS-LFA, the adversary builds an accurate link map and flood the fragile links. If the fragile links can be identified before DDoS-LFA occurs, one just need to monitor certain links instead of all links, thus increasing detection efficiency and reducing data storage.

In this module, each C-NODE first initializes an empty source space (SS) and an empty destination space (DS), and then periodically reads packet information including the source and destination IPs, followed by adding the IPs to corresponding spaces. Next, each C-NODE calculates link weights in the manner described in Section IV-B, and updates their values on blockchain. The blockchain ranks them in a decreasing order and picks a few links with the highest weights as fragile links. This result is listened by all C-NODEs, then they can deploy the detection and mitigation strategies.

D. Link Monitoring and Feature Extraction Module

In this module, C-NODEs monitor the fragile links and extract features to discover abnormality and identify bots. Although the traffic launched by the adversary resembles legitimate traffic, the features of links and traffic sources are still different from those in normal network environment. On one hand, the link features of a local network are highly time-dependent in normal environment. However, the attack is sudden and unpredictable. To indicate whether the congestion is caused by DDoS-LFA, we adopt 3 link features, including Link Bandwidth Utilization (LBU), Packet Count (PKT) and Round-Trip Time (RTT). When LBU, PKT and RTT unconventionally increase, it should be alert for attacks. On the other hand, in normal network, the traffic triggered by a generic source is ruleless. However, during the attack, the adversary installs similar programs on bots, so the flows sent by those bots are correlated. We use 4 standard deviations to identify the bots: Standard Deviation of average Size of Packets sent (SDSP), Standard Deviation of Packets sent (SDP), Standard Deviation of Bytes sent (SDB) and Standard Deviation of Time interval of packets sent (SDT). When the packets sent by different sources show similar characteristics, we mark these sources as bots.

To ensure the correctness of raw data mentioned above, we use blockchain to store them. In the CREDIT network, each record must be confirmed by all C-NODEs before being written on blockchain. The consensus algorithm picks out a C-NODE to package all transactions and generate the next

block. This is a decentralized process, since the node chosen as the bookkeeping node is completely random in each round, making it challenging for an attacker to tamper data.

E. Attack Detection Module

This module is used to detect DDoS-LFA. The traffic features change in an irregular way during DDoS-LFA, which is quite different from features in normal network. Therefore, we can find abnormality through detecting the change point. The selected features are shown below:

- LBU: When congestion occurs, LBU increases sharply, which can be easily calculated through dividing the receiving/sending packet rate by link bandwidth.
- PKT: PKT can be easily obtained from receiving/sending interface. It will rise when the attack occurs.
- RTT: By sending ping packets to neighbors periodically, RTT can be obtained. It will increase when links are blocked. Since the ping packet size is small, it won't affect the link bandwidth.

The running process of this module is divided into the start-up and smooth-running phases. In start-up, the C-NODE uses the LBU (or PKT) obtained from blockchain to train a credible prediction model, so that it can predict the link features of a normal network. In smooth-running, the C-NODE utilizes the well-trained model to make prediction. Once the extracted LBU (or PKT) is significantly different from the predicted data, an LBU (or PKT) alarm will be raised. If both the LBU and PKT alarms are raised and the RTT value goes up, an attack alarm is triggered. The detection and prediction process is elaborated in Section IV-C.

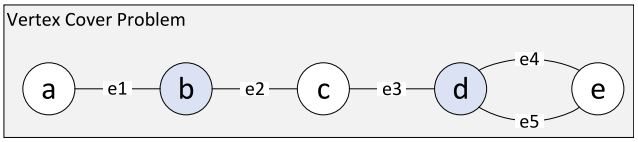
F. Source Identification Module

This module is used to identify suspicious bots. The packet sending rates, receiving interval and bandwidth consumption are semblable during DDoS-LFA, because the adversary launches traffic at a constant rate with an army of bots. Since standard deviation measures the dispersion of a set of values, a low standard deviation indicates that the values tend to be close to the average value, while a high standard deviation indicates that the values are spread out over a wide range. Bots will show similar characteristics, which means they have similar standard deviations and the values of these standard deviations are small. Therefore, we choose standard deviation to facilitate the identification of bots.

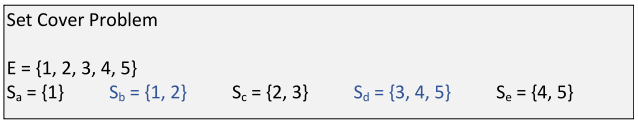
When an attack alarm is triggered, each C-NODE calculates the SDSP, SDP, SDB and SDT of all sources. Then, the potential bot list can be obtained by analyzing the similarity of pairwise sources. After that, each C-NODE uploads its list to the blockchain, and the blockchain will take the intersection of all source lists as the final result. The calculation process is elaborated in Section IV-D.

G. Attack Mitigation Module

Given the final list of suspicious sources, the blockchain delivers it to all C-NODEs, which will mitigate congestion according to the following rules: If the C-NODE linked to the congested link has an alternate link with low LBU, it reroutes the traffic sent by the suspicious sources to the alternative link. Otherwise, it drop the packets. By employing these rules, the congestion can be controlled without affecting the packets from normal sources.



(a) VCP



(b) SCP

Fig. 5. Vertex Cover Problem and Set Cover Problem.

IV. Key Algorithms of CREDIT

A. Node Selection Algorithm Based on Greedy Strategy

1) Problem Definition: Recalling Section III-B, we aim to select a few nodes as C-NODEs, so as to save computing and storage resources of routers, as well as minimize the amount of content stored in the blockchain. The selection strategy must guarantee that each link is connected to at least 1 C-NODE. The selection problem is a Vertex Cover Problem (VCP). Take Fig. 5(a) as an example, there are 5 vertexes and 5 edges. The goal of VCP is to find out the minimum set of nodes that can cover all links. It is observed that choosing b and d is the optimal solution.

A VCP can be solved by formulating it into a Set Cover Problem (SCP), which contains a set of elements, $E = \{e_1, e_2, \ldots, e_n\}$, and a set of subsets of E, $S = \{S_1, S_2, \ldots, S_m\}$, where $S_i = \{e_k, e_{k+1}, \ldots\}$ and the union of S_i will give E. The SCP is to find a collection C, which is the smallest subset of S and covers all elements in E. To translate VCP into SCP, each vertex n is regarded as a subset S_n, and the edge e_n linked to n is added to the corresponding subsets S_n. For example, as shown in Fig. 5, the vertex a and b are regarded as subset S_a and S_b, respectively. Both of them have an edge e_1, hence the element 1 is added to S_a and S_b. Repeat the above steps until all the edges are added to the corresponding subsets. At last, Fig. 5(a) is transferred to Fig. 5(b).

2) Selection Strategy: The SCP has a greedy solution, i.e., if E contains elements not covered by C, the set S_n which contains the largest number of elements should be obtained and added to C. We apply this solution to our node selection scheme.

The solution is illustrated based on a more complicated example in Fig. 6. Firstly, the edge nodes of the local network are marked as C-NODEs (n_3, n_{15}, n_{16} and n_{17}), which make up the initial CREDIT network and will participate in the later consensus process. Then, the smart contract calculates the number of neighbors of the remaining nodes (hereinafter

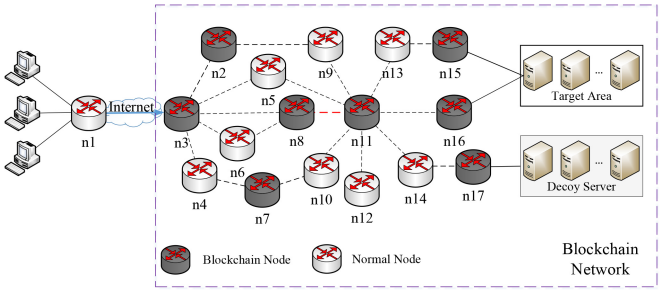


Fig. 6. C-NODE selection strategy.

referred to as degree, e.g., the degree of n_{11} is 8), and ranks them in a decreasing order based on their degrees. Next, the smart contract marks the node with the highest degree as C-NODE if its status is U-NODE, or ignores it otherwise (e.g., n_{11} is marked as C-NODE). Later, check C-NODE's neighbours from neighbour table. If the status of a neighbouring node is U-NODE and that node has at least one other neighbour with the status of C-NODE, then mark it as N-NODE (n_5 is a neighbour of n_{11} and is adjacent to another C-NODE n_3, hence n5 is categorized as an N-NODE). Similarly, if the status of a neighbour is C-NODE and it has only one neighbour which is a C-NODE, also classify it as an N-NODE (when both n_4 and n_7 are labeled as C-NODE, n_4 will be updated to a N-NODE). In other cases, the neighbour node is ignored, with its original status unchanged. Repeat the process until all remaining nodes have been selected and marked as C-NODE or N-NODE. Thus, no U-NODE will be left. Then, all the selected nodes have a connection to every link. We detail the strategy in Algorithm 1. Note that once a node is marked as C-NODE, it gets involved in the consensus process.

The consensus process consists of two steps. First, all the edge nodes, which are the initial C-NODEs, use Proof of Stake (PoS) for consensus. PoS is a widely used consensus mechanism in blockchain, which allocates the block creation rights based on stakes of the nodes rather than their computational power. Next, the smart contract selects new C-NODEs, and the selected nodes participate in the consensus process. After the selection process is finished, all the C-NODEs that have passed the consensus verification are obtained.

It has been proved that the greedy algorithm has a polynomial time complexity [51]. In the topology shown in Fig. 6, it only takes about 1 second to select C-NODEs.

B. Fragile Link Analysis Algorithm Based on Traffic Density

Since the links with high flow density are more likely to be attacked, the potential target links can be identified by calculating the flow density of all links. Then, monitoring module can be deployed on these links. The flow density of a link is defined as the number of flows that can be created through that link. It can be computed as follows.

In the time slot *t*, when a C-NODE *n* sends a flow to a port (i.e., to a link) or receives a flow from a port (i.e., from a link), it checks whether the destination IP matches the IPs of our protected zone. If it matches, go to the next step, which can

Algorithm 1 C-NODE Selection Strategy

Input:
Status Table (ST);
Neighbour Table (NT);
Output:
Final Status Table (FST): A table indicating the final status of all nodes;

$ST \leftarrow$ Edge nodes marked as C-NODEs;
Calculate degree of all nodes except for edge nodes;
Rank the nodes in a decreasing order based on their degree, pick the top one node *n*;
if $ST(n) ==$ U-NODE **then**
  $ST \leftarrow n$ marked as C-NODE;
end if
Check $n's$ neighbours list $NL_n \leftarrow NT$;
for each node m in NL_n **do**
  check $m's$ neighbours list $NL_m \leftarrow NT$;
  if $ST(m) ==$ U-NODE **then**
    for each node q in NL_m **do**
      if $ST(q) ==$ C-NODE **then**
        $ST \leftarrow m$ marked as N-NODE;
        break;
      end if
    end for
  else if $ST(m) ==$ C-NODE **then**
    if NL_m has only one element and $ST(e) ==$ C-NODE **then**
      $ST \leftarrow m$ marked as N-NODE;
    end if
  end if
end for
Repeat above process until all links are involved;
Return Final Status Table.

be divided into two cases. In one case, *n* is the sender. It will check whether the next-hop is a C-NODE. If not, it records the source IP and destination IP of the flow; Otherwise, it ignores it. In the other case, *n* is the receiver. It records the source IP and destination IP. Then the C-NODE adds those IPs to the Source Space (called SS_i, where i means link number) and Destination Space (called DS_i) respectively. At the end of this time slot, the weight of $link_i$ is calculated by

$$W_i = |SS_i| \times |DS_i|, \tag{1}$$

where $|SS_i|$ represents the cardinality of SS_i. Consider the example in Fig. 6, n_8 has three links, $link_{n_3-n_8}$, $link_{n_6-n_8}$ and $link_{n_8-n_{11}}$. In time slot *t*, n_8 receives three flows from $link_{n_3-n_8}$, the flows' IP space is as follows:

$$SS_{n_3-n_8} = \{32.0.0.25, 32.0.0.181\},$$
$$DS_{n_3-n_8} = \{240.0.0.9, 240.0.0.41\},$$

hence the weight of $link_{n_3-n_8}$ is 4. The greater the weight value of a link, the more likely it is to be a target link. We detail the strategy in Algorithm 2. By analyzing the flows, the potential target links can be quickly identified before the attack, hence improving attack detection efficiency.

Algorithm 2 Fragile Link Analysis Strategy

Input:
Link Table (LT);
C-NODE Table (CT): A table indicating all C-NODEs;
Flow Table (FT): A table indicating flows recorded by each C-NODE;
Output:
Fragile Link List (FL): A list of potential target links;

for each C-NODE *n* in *CT* **do**
  for each flow f in *FT* **do**
    n sends f to $link_i$ or receives f from $link_i$;
    if $f.destination ==$ protected zone **then**
      if n is the sender **then**
        if $ST(n.next\_hop) \neq$ C-NODE **then**
          $SS_i \leftarrow f.source$;
          $DS_i \leftarrow f.destination$;
        end if
      else
        $SS_i \leftarrow f.source$;
        $DS_i \leftarrow f.destination$;
      end if
    end if
  end for
end for
for each link *i* in *LT* **do**
  $W_i = |SS_i| \times |DS_i|$;
  $LT \leftarrow W_i$;
end for
Ranks *LT* in decreasing order based on *W*;
$FL \leftarrow LT(top\_l)$;
Return Fragile Link List.

After calculating link weights, the C-NODE uploads them to the blockchain, and the smart contract pick out the top_l links as potential target links for monitoring. This result is achieved by consensus algorithm among all C-NODEs, which is credible and tamper-resistant.

C. LFA Detection Algorithm Based on Link Features

We develops a credible deep learning method, called Credible Long Short-Term Memory (C-LSTM), to predict the LBU (link bandwidth utilization) and PKT (packet count) value of next time slot. Traditional LSTM is a kind of recurrent neural network [52], which performs well with time-varying sequences by selectively remember or forget some information. However, since LSTM uses historical data to predict the future, it is vulnerable to false past data. To guarantee that the module can still predict normal data even if an attack is occurring, we make certain modifications to improve the credibility of the input. We first define some symbols about LBU and PKT:

- LBU_i / PKT_i: Real value of the time slot *i*;
- LBU_i' / PKT_i': Predicted value of the time slot *i*;
- LBU_i^* / PKT_i^*: Modified value of the time slot *i*.

The symbols are stored on the blockchain to ensure the correctness. The prediction process based on C-LSTM can be divided into two phases: *start-up* and *smooth running*. In the following paragraph, we take the prediction process of LBU as an example.

1) Start-Up: In this phase, we collect the real-time data to train the model parameter of the prediction module. The training process is running based on the data batch denoted by $X = [X_1, X_2, \ldots, X_N]$, where the element $X_t = [LBU_t, LBU_{t+1}, \ldots, LBU_{t+\tau-1}]$ is a data sample of the actual LBU values from the time slot *t* to $t+\tau-1$, *N* is the number of samples used in each iteration, and τ is the length of a sample. Further, we define the label $Y = [Y_1, Y_2, \ldots, Y_N]$, where the element $Y_t = [LBU_{t+1}, LBU_{t+2}, \ldots, LBU_{t+\tau}]$ represents the actual values from the time slot $t+1$ to $t+\tau$. It is expected that the output value of the C-LSTM-based prediction module is approximately equal to the label value, their difference thus can be used to reflect the accuracy of the prediction.

LBU_t is the input of C-LSTM, $y_t = f(LBU_t, W)$ is the output value, where *W* is the model parameter that should be optimized in each iteration, and $f(\cdot)$ is the function of a C-LSTM cell. The closer y_t value is to the actual value LBU_{t+1}, the better C-LSTM is trained. Therefore, the goal of the training process is to reduce the difference between y_t and LBU_{t+1}, and the loss function (also acts as the objective function) can be defined as follows,

$$F(W) = \frac{1}{N}\sum_{t=1}^{N} L(f(X_t, W), Y_t), \tag{2}$$

where

$$L(f(X_t, W), Y_t) = \frac{1}{\tau}\sum_{i=t}^{t+\tau-1} (f(LBU_i, W) - LBU_{i+1})^2. \tag{3}$$

We use the same way to train model for PKT prediction. Note that in the CREDIT network, all the nodes should share the same model so that the prediction result will not be affected no matter which node is selected to conduct prediction.

2) Smooth Running: With successfully trained C-LSTM, CREDIT system starts the attack detection in this phase. The prediction performance of C-LSTM is greatly affected by the input data. Using the real LBU as input during DDoS-LFA will lead to an unfaithful result. To guarantee that the model can make correct prediction when the attack occurs, we make modification to the real LBU. In the time slot *t*, the winning C-NODE first calculates LBU_t, and then fetches the predicted value LBU_t' from the block of the time slot $t-1$. The modified value is calculated by

$$LBU_t^* = \eta LBU_t + (1-\eta)LBU_t', \tag{4}$$

where $\eta \in [0,1]$ represents the trustworthiness of LBU_t. If the network is more likely to be normal and LBU_t is thus credible, η should be set larger. When the attack is occurring and LBU_t is no longer trustworthy, η should be set smaller. The value of η is adaptively updated as follows,

$$\eta = \begin{cases} 1 & if\ \frac{\delta_t}{LBU_t'} \leq \mu, \\ \mu \times \frac{LBU_t'}{\delta_t} & otherwise, \end{cases} \tag{5}$$

Algorithm 3 LFA Detection and Link Feature Prediction (LBU) Strategy

Input:
LBU'_t;
$[LBU^*_{t-1}, ..., LBU^*_{t-\tau+1}]$;
Fragile Link List (FL);
μ: Control factor of sensitive degree;
ϵ_t: Number of times the abnormality occurs;
ε: Threshold of whether an attack occurs;
Output:
$block_t$: Latest block;
A_{LBU}: Attack alarm of LBU;

All C-NODEs compete for packaging right. The winning C-NODE will do:
  for each link l in FL **do**
    Calculate LBU_t of l;
    $\delta_t = |LBU_t - LBU'_t|$
    if $\frac{\delta_t}{LBU'_t} \leq \mu$ **then**
      $\eta \leftarrow 1$;
      $\epsilon_t \leftarrow 0$;
    else
      $\eta \leftarrow \mu \times \frac{LBU'_t}{\delta_t}$;
      $\epsilon_t \leftarrow \epsilon_t + 1$;
    end if
  end for
  if $\epsilon_t \geq \varepsilon$ **then**
    Raise an attack alarm A_{LBU};
    $\epsilon_t \leftarrow 0$;
  end if
  $LBU^*_t \leftarrow \eta LBU_t + (1-\eta)LBU'_t$;
  $\tilde{X}_t \leftarrow [LBU^*_t, LBU^*_{t-1}, ..., LBU^*_{t-\tau+1}]$
  $LBU'_{t+1} \leftarrow clstm(\tilde{X}_t, W)$;
  $block_t \leftarrow LBU_t$, LUB^*_t, LBU'_{t+1}.

where $\delta_t = |LBU_t - LBU'_t|$ is the deviation and $\mu \in [0, 1]$ is an attack threshold. The value of δ_t is used to determine whether an attack is occurring, since LBU_t approximately equals LBU'_t in a normal network. If $\frac{\delta_t}{LBU'_t} \leq \mu$, the network is secure and η is set as 1. Otherwise, η is inversely proportional to δ_t. With LBU^*_t, we can guarantee the correct input of C-LSTM, so that it can continue making right prediction.

The winning C-NODE takes the τ modified values $\tilde{X}_t = [LBU^*_t, LBU^*_{t-1}, \ldots, LBU^*_{t-\tau+1}]$ as the input of C-LSTM to predict LBU'_{t+1}. The process can be defined by

$$LBU'_{t+1} = clstm\Big(\tilde{X}_t, W\Big). \tag{6}$$

To improve the fault tolerance, an indicator ϵ_t is used to keep track of the number of times that δ_t exceeds the threshold. When δ_t exceeds the threshold consecutively, CREDIT will regard this phenomenon as the omen of attack.

Algorithm 3 describes the LBU prediction process in detail. When ϵ_t exceeds the threshold ε, it raises an attack alarm of LBU. Note that we use the same way to predict PKT. When CREDIT raises both LBU alarms and PKT alarms, it checks the RTT value. If the RTT value rises abnormally, CREDIT determines that an attack is in progress. There is not a central node in CREDIT. In each time slot, the bookkeeping node selected by the consensus algorithm implements the detection mechanism. This process is completely random. Therefore, unless the attacker has the ability to know the selected node or has enough computing power to control the majority of nodes in the network, it is difficult to destroy the detection system.

D. Bots Identification Algorithm Based on Source Features

When the C-NODE raises the attack alarm, DDoS-LFA Mitigation is triggered. In Section III we elaborate that the packets sent by botnets have similar size and rates. If two hosts send traffic with similar characteristics, they're probably controlled bots. Therefore, we can estimate the randomness of traffic sent by various sources and identify the DDoS-LFA bots.

For each source, we select 4 standard deviations to analyze. The symbols used are as follows.

- $bytes_i$: bytes sent by a source during the time slot i;
- $pakcets_i$: packets sent by a source during the time slot i;
- $time_i$: time interval of 2 packets sent by a source.

SDSP (standard deviation of average size of packets sent): Bots continually send packets with similar size, hence result in a small SDSP. Its computational formula is as follows,

$$AVG_{sp} = \frac{1}{t}\sum_{i=1}^{t}\frac{bytes_i}{packets_i}, \tag{7}$$

$$SDSP = \sqrt{\frac{\sum_{i=1}^{t}\Big(\frac{bytes_i}{packets_i} - AVG_{sp}\Big)^2}{t-1}}. \tag{8}$$

SDP (standard deviation of packets sent): Bots continually send traffic with the similar amount of packets in time intervals, thus result in a small SDP. Its computational formula is as follows,

$$AVG_p = \frac{1}{t}\sum_{i=1}^{t} packets_i, \tag{9}$$

$$SDP = \sqrt{\frac{\sum_{i=1}^{t}\big(packets_i - AVG_p\big)^2}{t-1}}. \tag{10}$$

SDB (standard deviation of bytes sent): Bots continually send traffic with the similar size in time intervals, thus result in a small SDB. Its computational formula is as follows,

$$AVG_b = \frac{1}{t}\sum_{i=1}^{t} bytes_i, \tag{11}$$

$$SDB = \sqrt{\frac{\sum_{i=1}^{t}(packets_i - AVG_b)^2}{t-1}}. \tag{12}$$

SDT (standard deviation of time interval of packets sent): Bots continually send traffic with a fixed period, thus result in a small SDT. Different from the first three metrics, it does not count the packets or bytes, but uses the sending time interval of several packets. Its computational formula is as follows,

$$AVG_t = \frac{1}{N}\sum_{i=1}^{N} time_i, \tag{13}$$

Algorithm 4 Suspicious Source Identification Strategy

Input:
SD List: [SDSP, SDP, SDB, SDT], calculated standard deviations of various sources;
μ: Threshold of whether 2 sources are bots;
Output:
Suspicious Source List (SL): A list of suspicious sources;

Gap List $\leftarrow \emptyset$;
for each item in SD List **do**
  if item.SDs in range (G1, G7) **then**
    Gap List $\leftarrow$ item;
  end if
end for
for each element e_1 in *Gap List* **do**
  for each element e_2 in *Gap List* **do**
    if $e_1 \neq e_2$ **then**
      Calculate S_{e_1,e_2};
      if $S_{e_1,e_2} \leq \mu$ **then**
        SL $\leftarrow e_1$;
        SL $\leftarrow e_2$;
      end if
    end if
  end for
end for
Return Suspicious Source List.

$$SDT = \sqrt{\frac{\sum_{i=1}^{N}(time_i - AVG_t)^2}{N-1}}. \tag{14}$$

The four metrics are normalized to the range of [0, 1]. To make it easier to obtain the similarity of different sources, we divide the range into 10 *Gaps*, which are [0,0.1), $[0.1, 0.2), \ldots, [0.9, 1]$, respectively, and mark them as $(G1, G2, \ldots, G10)$. Then put the metric to the appropriate *Gap*. For example, if a source's four metrics are (0.15, 0.22, 0.67, 0.8), its metrics are marked as $(G1, G2, G6, G8)$.

After calculating the 4 metrics, each C-NODE first seeks out the sources whose 4 metrics are between $G1$ and $G7$ because the randomness of these sources is small. Then it calculates the similarity of pairwise sources by

$$S_{x_1,x_2} = \sqrt{\frac{\sum_{i=1}^{4}\left(Gap_i^{x_1} - Gap_i^{x_2}\right)^2}{4}}, \tag{15}$$

where Gap_i^x means the x-th source's i-th metric. If S is less than the threshold, the two sources x_1 and x_2 are likely to be bots. To save the calculation time, it no longer calculates S_{x_1,x_3} if both S_{x_1,x_2} and S_{x_2,x_3} are small, which means that if x_1 and x_2 are alike, x_2 and x_3 are alike, it estimates that x_1 and x_3 are alike. The detailed calculation scheme is shown in Algorithm 4. After each C-NODE finishes calculation and gets the suspicious sources, the smart contract will take the intersection of all the sources, which ensures that some legitimate sources will not be mistakenly identified as malicious ones.

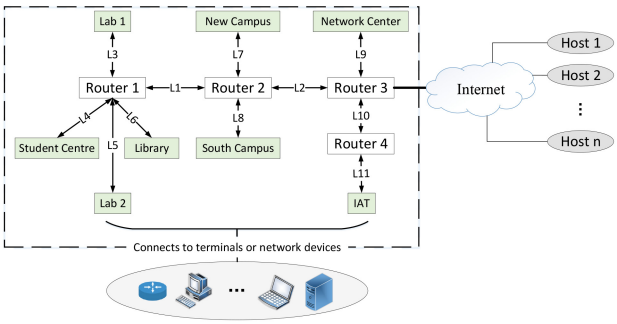


Fig. 7. Network topology for our experiments.

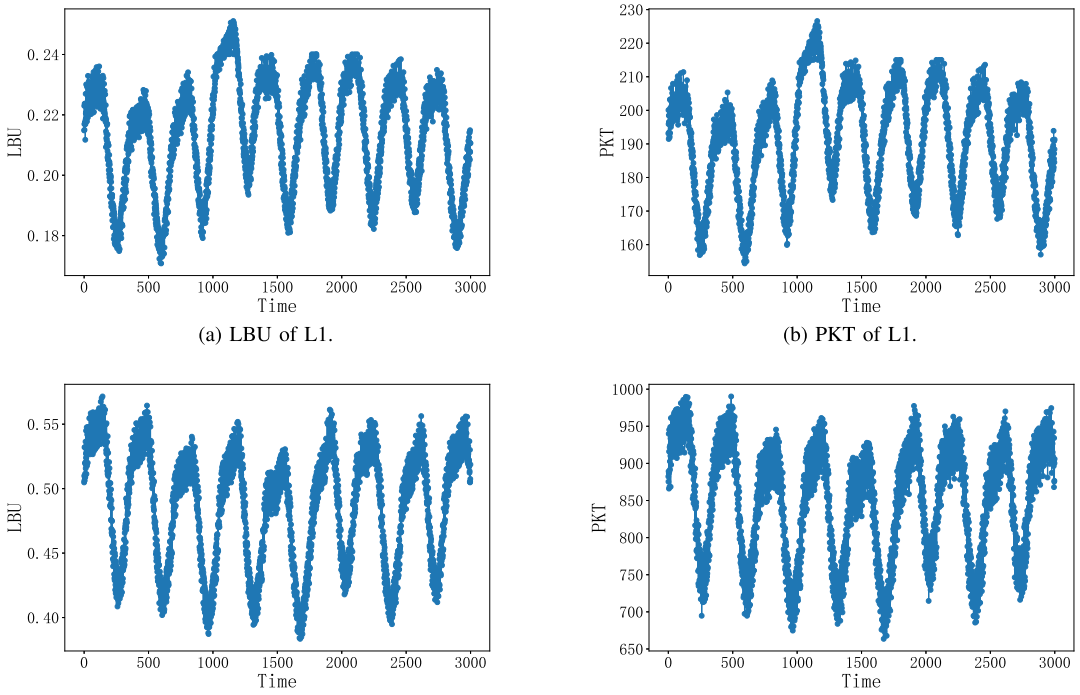
V. Performance Evaluation

In this section, we conduct experiments to evaluate the performance of CREDIT. First, we introduce our experimental environment, including building test range, generating normal traffic and attack traffic, as well as displaying the result of CREDIT Node Selection and Fragile Link Analysis. Then, we evaluate the performance of the proposed algorithms and compare them with some existing works. At last, we interpret the overhead of the CREDIT system.

A. Experimental Environment

1) Testbed: Fig. 7 is the topology of our testbed, which is the campus network topology of the University of Science and Technology of China (USTC). In Fig. 7, the green nodes are routers that are connected to other terminals or network devices. For example, the node of New Campus denotes the core router in the New Campus of USTC, which is connected to thousands of personal computers and cell phones. In our experiment, this node is re-built via a virtual machine using the system of ubuntu 16.04 with 8-core CPU, 16GB memory, and 40GB disk space. The connected devices are simulated by the captured traffic shown in ns4.ustc.edu.cn. The white nodes are primary routers that gather traffic, and router 3 is the gateway directly connected to the Internet. The gray nodes are hosts that send traffic to the campus network, where $n = 6$ and hosts 1, 2, ..., 6 are CREATE2(IPv6), CREATE(IPv4), ChinaTelCom, ChinaUnionCom, ChinaMobileCom and CSTNET, respectively.

We build the blockchain system based on Ethereum in CENI platform. On this platform, users can create real nodes (virtual machines or VSR routers) located on different regional servers. They can configure system settings, CPUs, storage, network segments, and establish connections between nodes by configuring bandwidth, packet loss rate, latency, and other parameters. Tcpdump and "/proc/net/dev" are used for nodes to monitor network traffic. Tcpdump is a packet capture and analysis tool that can capture packets in real-time, providing information about the time, type, and length of the received packets on the network interface. "/proc/net/dev" is a tool for reading network statistics information, allowing real-time monitoring of traffic and providing information about the number of packets received, bytes transferred, and packet loss of the network within a certain period. The iptables is used to filter packets. When a packet enters the network card, iptables



(a) LBU of L1. (b) PKT of L1.

(c) LBU of L2. (d) PKT of L2.

Fig. 8. The LBU and PKT of some links.

will processes the packet based on the method defined by the rules.

2) Generation of Traffic: We extract the real world traffic of USTC campus network and apply it as the normal traffic in the testbed. The LBU and PKT of some links are shown in Fig. 8. We also generate attack traffic to flood the link. We define 2 variables, *host-server pair* (*pair* for short) to characterize the scale of the attack, λ to represent the incremental speed at which each host sends packets to the server, that is, each host will increase the sending traffic at the speed of λ until the aggregated flows are slightly higher than the link bandwidth. For example, if $pair = 100$, $\lambda = 4$, it means that there are 100 host-server pairs in total. Each host sends traffic at the rate of 4 Kbps until the target link congests.

3) CREDIT Node Selection Result: In our testbed, the green nodes, Router 1 and Router 3 are C-NODEs, who are responsible for detecting and mitigating the attack.

4) Fragile Link Analysis Result: In our testbed, L1 and L2 are recognized as fragile links. Router 1 and Router 3 will conduct detection and mitigation schemes in these 2 links.

5) Other Parameters: The learning rate and epoch of LSTM are set as 0.006 and 500 respectively. The difficulty in the genesis block is set as 0x400000. When configuring a private blockchain, the difficulty value can be adjusted according to the needs.

6) Workflow: The CREDIT deployed on our testbed works as follows: Firstly, the network administrator performs initialization process, selects the edge nodes marked in green in Fig. 7 as C-NODEs, and generate the status table, neighbour table and link table on smart contract of the blockchain. Then, with the help of CREDIT Node Selection module, the blockchain selects Router 1 and Router 3 to add them to the CREDIT nodes, which are responsible for detecting and mitigating attacks. Next, the CREDIT nodes conduct Fragile Links Analysis module, pick out L1 and L2 as fragile links to be monitored. Afterwards, all C-NODEs periodically extract features of L1 and L2, upload them to blockchain and utilize these features to identify potential attacks. Once DDos-LFA attacks from identified bots are detected on L1 and L2, the CREDIT nodes connected to them (i.e., Router 1 and Router 3) mitigate congestion by reroute or drop packets.

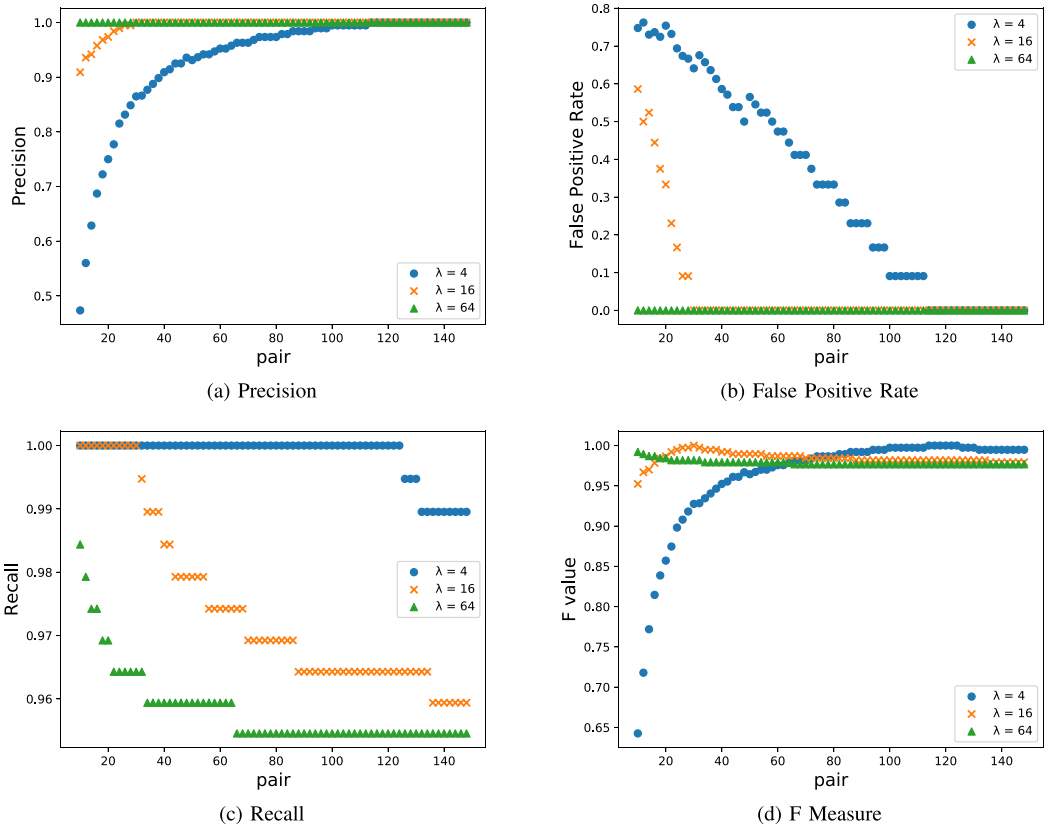
B. Existing Work

We compare the performance of CREDIT with Balance [53], Woodpecker [14] and LFADefender [54]. The existing algorithms are introduced as follows:

1) Balance: It uses SDN switches to collect 9 metrics of each source and calculates the Shannon's entropy. If the entropy is lower than the attack threshold, Balance determines that the congestion is due to DDoS-LFA. Then Balance selects the sources with similar characteristics and records them as bots.

2) Woodpecker: It evaluates the importance of links on the current flow path, and calculates the weight of each link when congestion occurs. If the weight is greater than the attack threshold, Woodpecker decides that there is an attack. Then Woodpecker identifies the IP addresses which appear on several congested links as bots.

3) LFADefender: It first analyzes the links that may be regarded as targets, then records the number of traceroute packets sent by each source and determines whether the source is a bot based on the attack threshold.



(a) Precision (b) False Positive Rate

(c) Recall (d) F Measure

Fig. 9. Precision, false positive rate, recall and F measure varying with λ and *pair*.

C. Performance of CREDIT

1) Result of DDoS-LFA Detection: We adopt the precision (P), false positive rate (FPR), recall (R) and F measure (F) to measure the detection accuracy of CREDIT. The formulas are as follows:

$$P = \frac{TP}{TP + FP}, \tag{16}$$

$$FPR = \frac{FP}{FP + TN}, \tag{17}$$

$$R = \frac{TP}{TP + FN}, \tag{18}$$

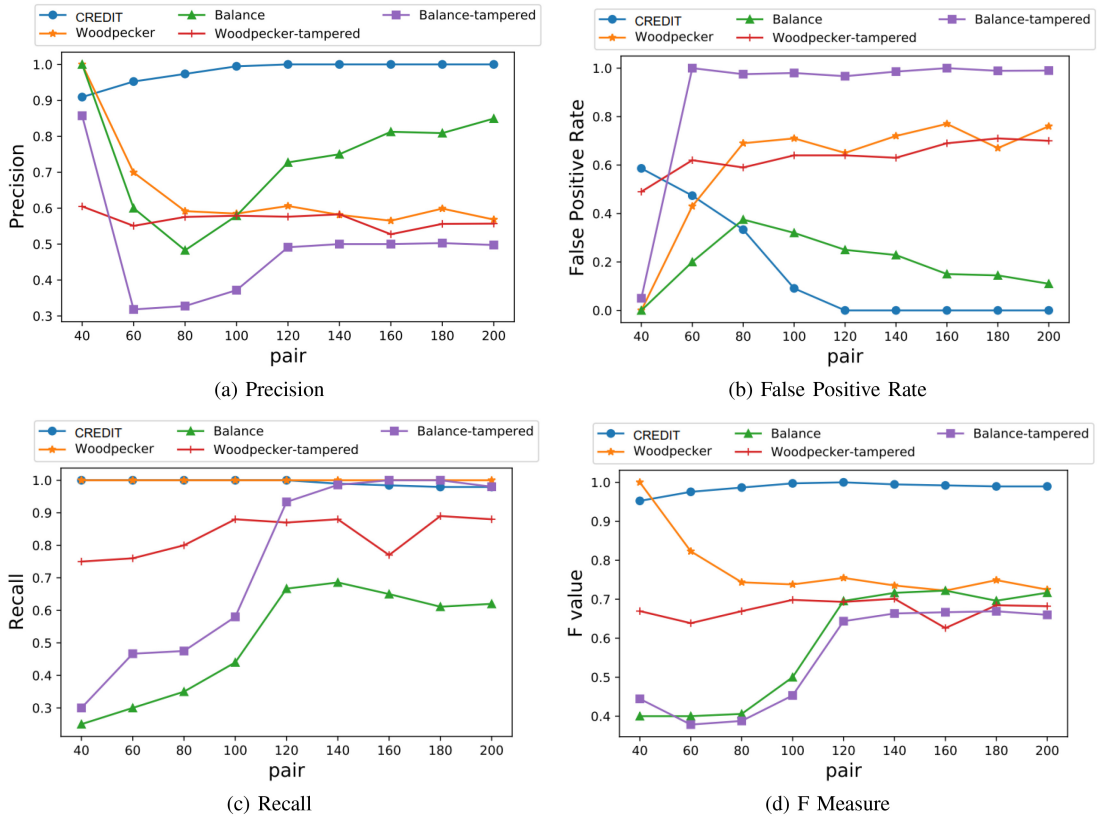
$$F = \frac{2 \times TP}{2 \times TP + FP + FN}, \tag{19}$$

where TP means the system correctly discerns an abnormal event as attack, FP means the system misjudges a legal event as attack, TN means the system accurately recognizes an abnormal event as legitimate traffic, and FN means the system falsely recognizes an abnormal event as legitimate traffic.

We use different (λ, *pair*) to test the 4 metrics of CREDIT. Fig. 9(a) shows the change pattern in precision over λ and *pair*. When the λ value is settled, the precision rises with the increase of the *pair* value. This is because the larger the *pair* value is, the more attack flows there are. Hence the traffic will increase in a short period of time, making it easier to detect the attack. Analogously, when the *pair* value is settled, the larger the λ value is, the larger the traffic sent by each *pair* is, making it easier to find out exceptions. In addition, precision eventually approaches 100%, indicating that the CREDIT detection algorithm has a high accuracy. Fig. 9(b) is false positive rate varying with λ and *pair*. A high FPR means that the system is likely to mistakenly identify a normal flow as an attacked one. When the *pair* value is low, FPR is high when $\lambda = 4$. This is because when the number of hosts is small, link congestion occurs after a long period of time. However, CREDIT misjudges the traffic as an attack before congestion, resulting in a high FPR. Fig. 9(c) is the change in recall over λ and *pair*. A low recall indicates that the system is unresponsive, which means that it takes longer to detect the attack. As can be seen from the result, when the λ value is small, the recall is very high. The reason is that the abnormal traffic lasts for a long time, so CREDIT can quickly detect attacks. In addition, even if the λ value is large, recall can maintain 90% and above, indicating that CREDIT system is sensitive to attacks. Fig. 9(d) is F measure varying with λ and *pair*. F value goes up with the increase of the *pair* value, and finally tends to 100%. Similar to precision, when the *pair* and λ value is large, the traffic is more likely to appear abnormal, so it is easier to be detected by CREDIT.

LFA adversary uses many bots to send low-rate traffic to flood target links, which means that the λ value is small while the *pair* value is large in the attack. As can be seen from the experimental results, the CREDIT system has high precision, recall and F1 measure, as well as low false positive rate.

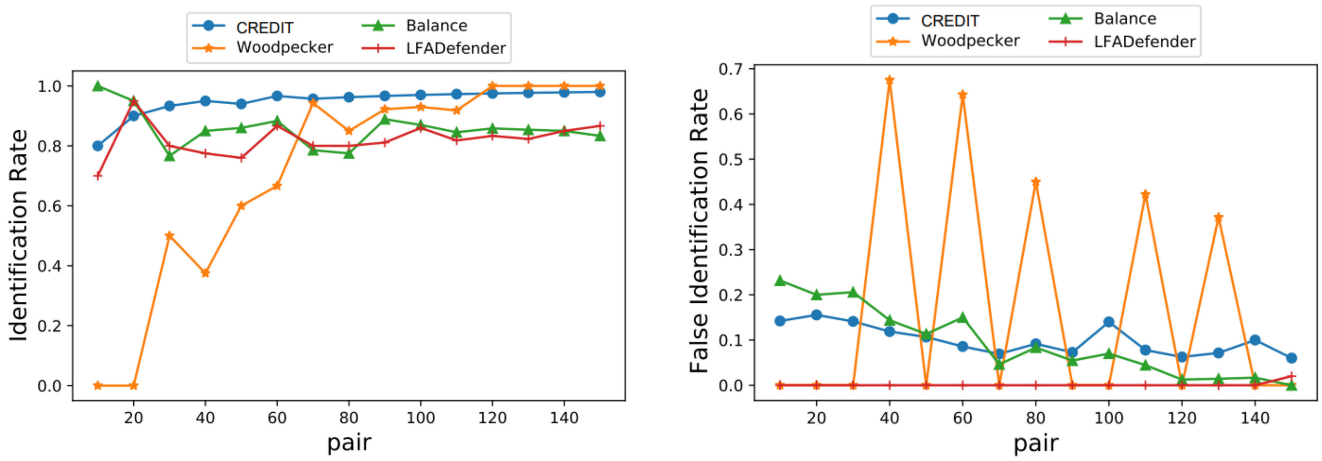
We test the detection performance between CREDIT, Balance and Woodpecker. Since the traffic rate sent by the DDoS-LFA attacker is low, we set the λ value to 4kbps in this experiment, and evaluate the influence of the *pair* value



(a) Precision (b) False Positive Rate

(c) Recall (d) F Measure

Fig. 10. Comparison of precision, false positive rate, recall and F measure between CREDIT, Balance and Woodpecker.

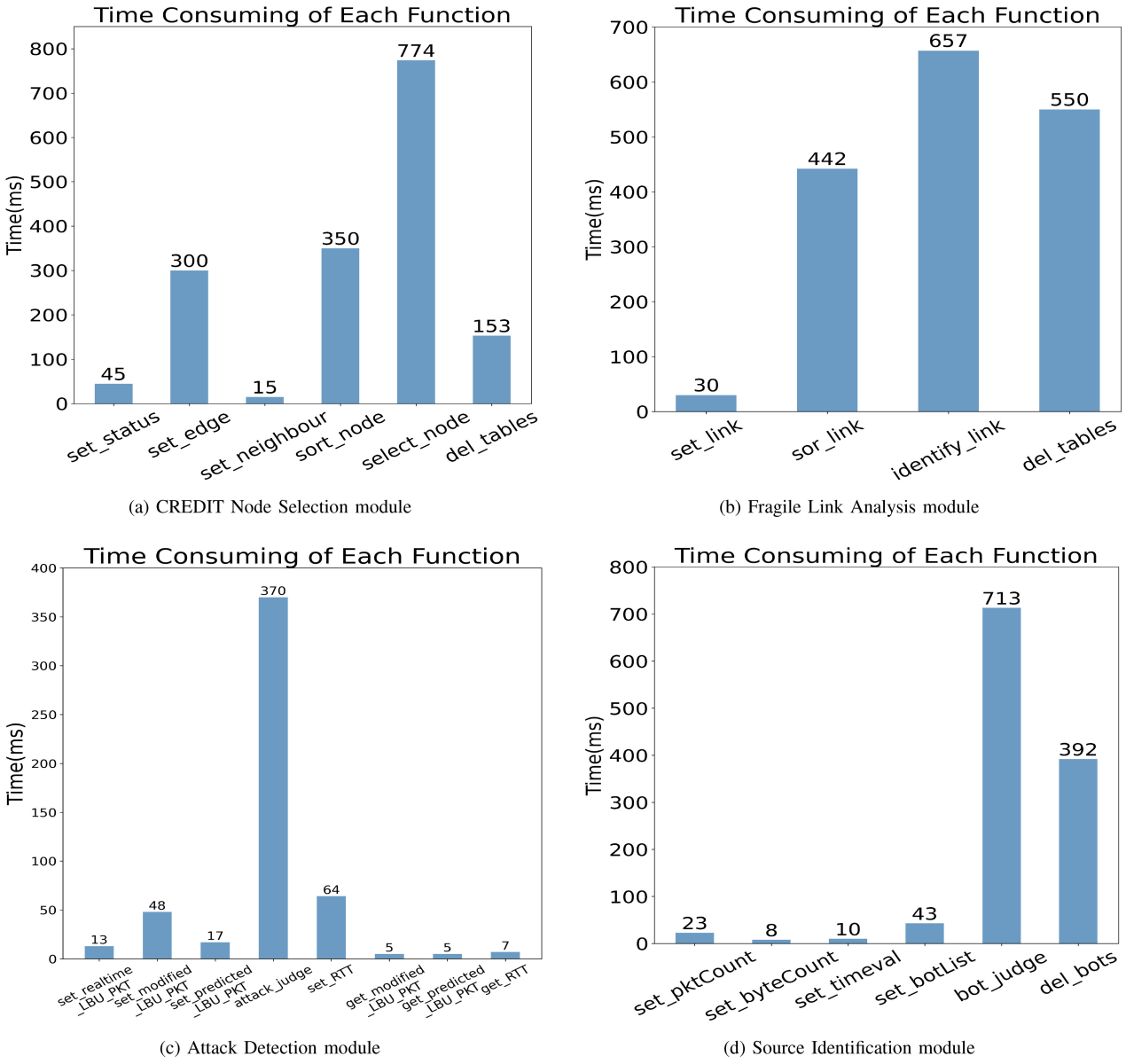


(a) Identification rate (b) False identification rate

Fig. 11. Identification rate and false identification rate between CREDIT, Balance, Woodpecker and LFADefender.

on the 3 mechanisms. In addition, a small amount of data used by Balance and Woodpecker is tampered randomly in this experiment to evaluate the effect of data reliability on the detection mechanism.

Fig. 10(a) shows the comparison of precision. When the *pair* value is small, the effect of Balance and Woodpecker is slightly better than that of CREDIT. However, when the *pair* value increases, the precision of CREDIT is significantly higher than that of Balance and Woodpecker. Besides, when we randomly tamper with the data used by Balance and Woodpecker, their precision drop dramatically. Fig. 10(b) shows the comparison of false positive rate. FPR of CREDIT is higher than any other schemes when the *pair* value is small. When the *pair* value increases, the FPR of CREDIT tends to 0, while that of Balance and Woodpecker keep large. Besides, if the data is tampered with, the FPR of Balance has a sharp increase, and the FPR of Woodpecker keeps large too, which indicates that the 2 mechanisms have a high demand for data reliability, while the CREDIT is not affected by tampering due to the use of blockchain. Fig. 10(c) and Fig. 10(d) show the comparison between recall and F measure, and it can be seen that the



(a) CREDIT Node Selection module

(b) Fragile Link Analysis module

(c) Attack Detection module

(d) Source Identification module

Fig. 12. Time cost of functions in each module.

detection effect of CREDIT is superior than Balance and Woodpecker.

2) Result of Source Identification: We adopt the attack source identification rate (IR) and false identification rate (FIR) to evaluate the performance. High IR and low FIR indicate that the system has good attack source identification performance and can discard attack traffic in time to alleviate congestion. The calculation formulas are as follows:

$$IR = \frac{IB}{TB}, \tag{20}$$

$$FIR = \frac{MIH}{TH}, \tag{21}$$

where IB means the number of identified bots, TB means the total number of bots, MIH means the number of normal hosts misjudged as bots, and TH means the total number of normal hosts.

Fig. 11(a) and Fig. 11(b) show the comparison results of attack source identification between CREDIT, Balance, Woodpecker and LFADefender. It can be seen from the results that the IR of 4 schemes reach over 80%, and the performance of CREDIT is better than that of Balance and LFADefender, but is worse than that of Woodpecker when *pair* value is large. The reasons are as follows: Balance extracts nine features, analyzes the possible attack sources for each feature, and takes intersection of the suspicious sources to obtain the bots list. This method may omit some bots. Because there may be a malicious source that exhibits the characteristics of a bot under one feature, and display the characteristics of a normal host under another. LFADefender uses the number of traceroute packet to identify attack sources. While the adversary can deliberately reduce the number of packets to avoid being found out. Therefore, this method is hard to achieve 100% accuracy. Woodpecker recognizes the IP addresses that simultaneously

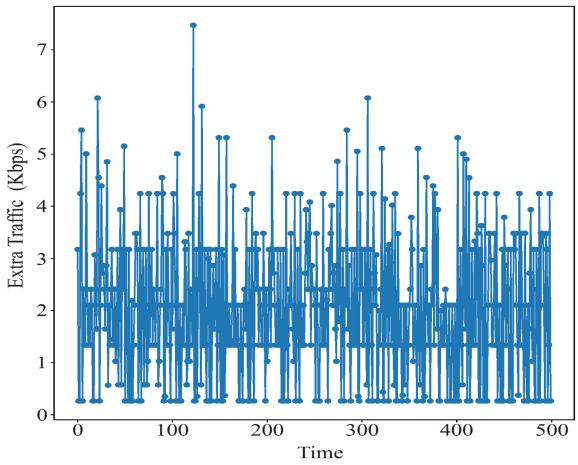


Fig. 13. Extra traffic generated by blockchain.

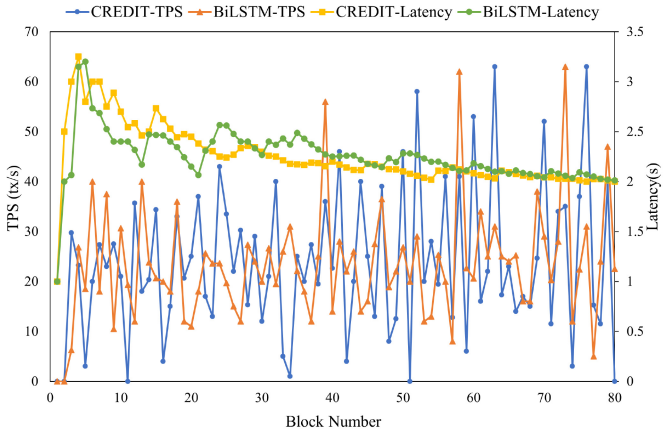


Fig. 14. Comparison of TPS and blocks forming latency between CREDIT and BiLSTM.

appears in many congested links as bots. This method is effective when there are numerous sources. Hence the result of Woodpecker shows a significant rise. Although the IR of Woodpecker is higher than that of CREDIT, its FIR shows a bad performance, which means Woodpecker is likely to regard legitimate users as bots. Based on the 2 results, we conclude that CREDIT has a good effect on identifying bots.

3) Overhead: We test the execution delay of each function of algorithm, as illustrated in Fig. 12. Execution time of each algorithm is primarily consumed by functions that involve multiple iterations of loops or complex computations, such as the select_node function in the node deployment module, which consumes 774 milliseconds of latency, the sort_link function in the link analysis module with a delay of 442 milliseconds, and the identify_link function taking 657 milliseconds. Additionally, the attack_judge function in the attack detection module and bot_judge function in the attack source identification module consume 370 milliseconds and 713 milliseconds respectively. These functions require numerous steps to execute and consequently result in longer processing times. Another significant factor contributing to execution time is data writing operations performed by set functions depicted in the figure; for instance, set_status within the node deployment module incurs a delay of 45 milliseconds. Such functions necessitate modification of contract variables which adds additional processing time. Conversely, simple data reading tasks take minimal time as observed with get functions illustrated in this figure; for example, get_RTT within the attack detection module only requires 7 milliseconds for completion. These types of functions do not modify contract variables nor involve complex operations hence their shorter execution times are evident. Overall results demonstrate that even for highly time-consuming functions, they only require a few hundred milliseconds indicating efficiency achieved through our proposed algorithm.

The extra traffic introduced by blockchain is shown in Fig. 13. The synchronization of transactions and blocks imposes a small amount of traffic to the network, which is about 3kbps averagely, demonstrating that the impact on bandwidth is negligible. Moreover, during a scheme execution, 145 transactions are used to upload data to blockchain. The size of a block is around 640 bytes. The memory consumption for a scheme execution is approximately 90kb. For a machine with 1GB of memory, ideally, its memory can support over ten thousand detections. Therefore, the influence of the block size is negligible.

4) Performance of Blockchain: We compare the TPS of blockchain and latency in forming blocks between CREDIT and BiLSTM [55], as illustrated in Fig. 14. TPS means transactions per second which is an important metric for measuring the throughput of a blockchain. Latency indicates the time it takes for a block to be confirmed and added to the blockchain. It can be seen that the two algorithm have similar performance, the maximum TPS of blockchain is about 60 and the latency of forming blocks ranges from 1 second to 7 seconds.

VI. Conclusion and Future Work

In this paper, we propose a link flooding attack defense scheme, CREDIT, that leverages blockchain and LSTM to perform target-link analysis, DDoS-LFA detection, dangerous source identification and DDoS-LFA mitigation. By exploiting the fact of DDoS-LFA that the adversary will control multiple bots to send legitimate-like flows, leading to a sudden increase of LBU and PKT, we design the LSTM model so that all computing nodes in the CREDIT network can conduct detection schemes. Moreover, we use correlation analysis to infer the potential target links and suspected sources, so that the protected region can take timely precautions and mitigate the congestion. CREDIT is the first application of blockchain to the detection of complex DDoS-LFA, which can both detect and migrate suspicious traffic based on the blockchain, even in the presence of deceptive attackers. CREDIT is more reliable, flexible, effective and low-cost than traditional non-blockchain-based DDoS-LFA defense methods, which demonstrates the huge potential of the combination of blockchain and AI in the field of cybersecurity defense. We implement CREDIT in CENI with Ethereum to evaluate its efficiency and overhead. The result demonstrates that CREDIT can effectively detect DDoS-LFA with a low false positive rate and high detection rate, as well as low overhead.

REFERENCES

[1] H. Luo, Z. Liu, and S. Zhang, "Preventing DDoS flooding attacks with cryptographic path identifiers in future Internet," *IEEE Trans. Netw. Service Manage.*, vol. 19, no. 2, pp. 1690–1704, Jun. 2022.

[2] A. Praseed and P. S. Thilagam, "Multiplexed asymmetric attacks: Next-generation DDoS on HTTP/2 servers," *IEEE Trans. Inf. Forensics Security*, vol. 15, pp. 1790–1800, 2019.

[3] S. Kottler. "February 28th DDoS incident report." 2018. [Online]. Available: https://github.blog/2018-03-01-DDoS-incident-report/

[4] C. Cimpanu. "AWS said it mitigated a 2.3 Tbps DDoS attack, the largest ever." 2020. [Online]. Available: https://www.zdnet.com/article/aws-said-it-mitigated-a-2-3-tbps-ddos-attack-the-largest-ever

[5] M. Geva, A. Herzberg, and Y. Gev, "Bandwidth distributed denial of service: Attacks and defenses," *IEEE Security Privacy*, vol. 12, no. 1, pp. 54–61, Feb. 2014.

[6] J. Xing, J. Cai, B. Zhou, and C. Wu, "A deep ConvNet-based countermeasure to mitigate link flooding attacks using software-defined networks," in *Proc. Int. Symp. Comput. Commun. (ISCC)*, 2019, pp. 1–6.

[7] A. Studer and A. Perrig, "The coremelt attack," in *Proc. Eur. Symp. Res. Comput. Security*, 2009, pp. 37–52.

[8] M. S. Kang, S. B. Lee, and V. D. Gligor, "The crossfire attack," in *Proc. IEEE Symp. Security Privacy*, 2013, pp. 127–141.

[9] M. S. Kang and V. D. Gligor, "Routing bottlenecks in the Internet: Causes, exploits, and countermeasures," in *Proc. ACM SIGSAC Conf. Comput. Commun. Security*, 2014, pp. 321–333.

[10] C. Liaskos, V. Kotronis, and X. Dimitropoulos, "A novel framework for modeling and mitigating distributed link flooding attacks," in *Proc. 35th IEEE INFOCOM*, 2016, pp. 1–9.

[11] "DDoS impact survey reveals the actual cost of DDoS attacks." 2015. [Online]. Available: https://www.freebuf.com/articles/network/67107.html

[12] M. Prince. "The DDoS that knocked spamhaus offline (and how we mitigated it)." 2013. [Online]. Available: https://blog.cloudflare.com/the-DDoS-that-knocked-Spamhaus-offline-and-ho/

[13] L. Xue, X. Ma, X. Luo, E. W. W. Chan, T. T. N. Miu, and G. Gu, "LinkScope: Toward detecting target link flooding attacks," *IEEE Trans. Inf. Forensics Security*, vol. 13, pp. 2423–2438, 2018.

[14] L. Wang, Q. Li, Y. Jiang, X. Jia, and J. Wu, "Woodpecker: Detecting and mitigating link-flooding attacks via SDN," *Comput. Netw.*, vol. 147, pp. 1–13, Dec. 2018.

[15] L. Wang, Q. Li, Y. Jiang, and J. Wu, "Towards mitigating link flooding attack via incremental SDN deployment," in *Proc. Int. Symp. Comput. Commun. (ISCC)*, 2016, pp. 397–402.

[16] A. Aydeger, N. Saputro, K. Akkaya, and M. Rahman, "Mitigating crossfire attacks using SDN-based moving target defense," in *Proc. 41st Conf. Local Comput. Netw. (LCN)*, 2016, pp. 627–630.

[17] S. B. Lee, M. S. Kang, and V. D. Gligor, "CoDef: Collaborative defense against large-scale link-flooding attacks," in *Proc. 9th ACM Int. Conf. Emerg. Netw. Exp. Technol.*, 2013, pp. 417–428.

[18] T. Hirayama, K. Toyoda, and I. Sasase, "Fast target link flooding attack detection scheme by analyzing traceroute packets flow," in *Proc. IEEE Int. Workshop Inf. Forensics Security*, 2015, pp. 1–6.

[19] J. Takeuchi and K. Yamanishi, "A unifying framework for detecting outliers and change points from time series," *IEEE Trans. Knowl. Data Eng.*, vol. 18, no. 4, pp. 482–492, Apr. 2006.

[20] M. S. Kang, V. D. Gligor, and V. Sekar, "SPIFFY: Inducing cost-detectability tradeoffs for persistent link-flooding attacks," in *Proc. 23rd Annu. Netw. Distrib. Syst. Security Symp. (NDSS)*, 2016, pp. 53–55.

[21] P. Xiao, Z. Li, H. Qi, W. Qu, and H. Yu, "An efficient DDoS detection with bloom filter in SDN," in *Proc. IEEE Int. Conf. Trust, Security Priv. Comput. Commun.*, 2016, pp. 1–6.

[22] J. Zheng, Q. Li, G. Gu, J. Cao, D. K. Y. Yau, and J. Wu "Realtime DDoS defense using COTS SDN switches via adaptive correlation analysis," *IEEE Trans. Inf. Forensics Security*, vol. 13, pp. 1838–1853, 2018.

[23] W. Rafique, X. He, Z. Liu, Y. Sun, and W. Dou, "CFADefense: A security solution to detect and mitigate crossfire attacks in software-defined IoT-edge infrastructure," in *Proc. IEEE Int. Conf. High Perform. Comput. Commun.*, 2019, pp. 500–509.

[24] S. T. Zargar, J. Joshi, and D. Tipper, "A survey of defense mechanisms against distributed denial of service (DDoS) flooding attacks," *IEEE Commun. Surveys Tuts.*, vol. 15, no. 4, pp. 2046–2069, 4th Quart., 2013.

[25] M. N. M. Bhutta et al., "A survey on blockchain technology: Evolution, architecture and security," *IEEE Access*, vol. 9, pp. 61048–61073, 2021.

[26] M. Ul Hassan, M. H. Rehmani, and J. Chen, "Anomaly detection in blockchain networks: A comprehensive survey," *IEEE Commun. Survey Tut.*, vol. 25, no. 1, pp. 289–318, 1st Quart., 2023.

[27] D. Puthal, N. Malik, S. P. Mohanty, E. Kougianos, and G. Das, "Everything you wanted to know about the blockchain: Its promise, components, processes, and problems," *IEEE Consum. Electron. Mag.*, vol. 7, no. 4, pp. 6–14, Jul. 2018.

[28] S. Nakamoto. "Bitcoin: A peer-to-peer electronic cash system." 2008. [Online]. Available: https://git.dhimmel.com/bitcoin-whitepaper/

[29] H. Hellani, A. E. Samhat, M. Chamoun, H. El Ghor, and A. Serhrouchni, "On blockchain technology: Overview of bitcoin and future insights," in *Proc. IEEE Int. Multidiscip. Conf. Eng. Technol. (IMCET)*, 2018, pp. 1–8.

[30] C. K. Frantz and M. Nowostawski, "From institutions to code: Towards automated generation of smart contracts," in *Proc. 1st Int. Workshops Found. Appl. Self-Syst.*, 2016, pp. 210–215.

[31] J. Zhou, G. Feng, and Y. Wang, "Optimal deployment mechanism of blockchain in resource-constrained IoT systems," *IEEE Internet Things J.*, vol. 9, no. 11, pp. 8168–8177, Jun. 2022.

[32] P. P. Ray, D. Dash, K. Salah, and N. Kumar, "Blockchain for IoT-based healthcare: Background, consensus, platforms, and use cases," *IEEE Syst. J.*, vol. 15, no. 1, pp. 85–94, Mar. 2021.

[33] M. Baza, M. Nabil, M. Ismail, M. Mahmoud, E. Serpedin, and M. Ashiqur Rahman, "Blockchain-based charging coordination mechanism for smart grid energy storage units," in *Proc. IEEE Int. Conf. Blockchain*, 2019, pp. 504–509.

[34] O. B. Mora, R. Rivera, V. M. Larios, J. R. Beltrán-Ramírez, R. Maciel, and A. Ochoa, "A use case in Cybersecurity based in blockchain to deal with the security and privacy of citizens and smart cities Cyberinfrastructures," in *Proc. IEEE Int. Smart Cities Conf. (ISC)*, 2018, pp. 1–4.

[35] H. Li, R. Lu, L. Zhou, B. Yang, and X. Shen, "An efficient Merkle-tree-based authentication scheme for smart grid," *IEEE Syst. J.*, vol. 8, no. 2, pp. 655–663, Jun. 2014.

[36] G. Wood, "Ethereum: A secure decentralised generalised transaction ledger," Ethereum, Zug, Switzerland, Yellow Paper, 2013. [Online]. Available: http://gavwood.com/paper.pdf

[37] E. Androulaki et al., "Hyperledger fabric: A distributed operating system for permissioned blockchains," 2018, *arXiv:1801.10228*.

[38] M. Castro and B. Liskov, "Practical Byzantine fault tolerance," in *Proc. OSDI*, 1999, pp. 173–186.

[39] G. Zhang et al., "Reaching consensus in the Byzantine empire: A comprehensive review of BFT consensus algorithms," 2022, *arXiv:2204.03181*.

[40] M. Yin, D. Malkhi, M. K. Reiter, G. G. Gueta, and I. Abraham, "HotStuff: BFT consensus with linearity and responsiveness," in *Proc. ACM Symp. Princ. Distrib. Comput.*, 2019, pp. 347–356.

[41] G. Zhang and H.-A. Jacobsen, "Prosecutor: An efficient BFT consensus algorithm with behavior-aware penalization against Byzantine attacks," in *Proc. 22nd Int. Middleware Conf.*, 2021, pp. 52–63.

[42] G. Danezis, L. Kokoris-Kogias, A. Sonnino, and A. Spiegelman, "Narwhal and tusk: A DAG-based mempool and efficient BFT consensus," in *Proc. 17th Eur. Conf. Comput. Syst.*, 2022, pp. 34–50.

[43] W. Meng, E. W. Tischhauser, Q. Wang, Y. Wang, and J. Han, "When intrusion detection meets blockchain technology: A review," *IEEE Access*, vol. 6, pp. 10179–10188, 2018.

[44] N. Alexopoulos, E. Vasilomanolakis, R. N. Ivánkó et al., "Towards blockchain-based collaborative intrusion detection systems," in *Proc. Int. Conf. Crit. Inf. Infrastruct. Security*, 2017, pp. 107–118.

[45] B. Rodrigues, T. Bocek, A. Lareida, D. Hausheer, S. Rafati, and B. Stiller, "A blockchain-based architecture for collaborative DDoS mitigation with smart contracts," in *Proc. Int. Conf. Auton. Infrastruct., Manage. Security*, 2017, pp. 16–29.

[46] L.-Y. Yeh, J.-L. Huang, T.-Y. Yen, and J.-W. Hu, "A collaborative DDoS defense platform based on blockchain technology," in *Proc. 12th Int. Conf. Ubi Media Comput.*, 2019, pp. 1–6.

[47] A. E. Z. Houda, S. A. Hafid, and L. Khoukhi, "Cochain-SC: An intra- and inter-domain DDoS mitigation scheme based on blockchain using SDN and smart contract," *IEEE Access*, vol. 7, pp. 98893–98907, 2019.

[48] Z. Abou El Houda, A. Hafid, and L. Khoukhi, "Co-IoT: A collaborative DDoS mitigation scheme in IoT environment based on blockchain using SDN," in *Proc. IEEE Glob. Commun. Conf.*, 2019, pp. 1–6.

[49] G. Spathoulas et al., "Towards reliable integrity in blacklisting: Facing malicious IPs in GHOST smart contracts," in *Proc. IEEE (SMC) Int. Conf. Innov. Intell. Syst. Appl. (INISTA)*, 2018, pp. 1–8.

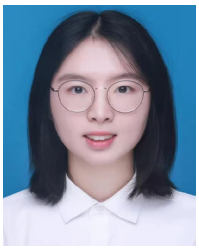
[50] M. Essaid, D. Kim, S. H. Maeng, S. Park, and H. T. Ju, "A collaborative DDoS mitigation solution based on Ethereum smart contract and RNN-LSTM," in *Proc. Asia Pac. Netw. Oper. Manag. Symp., Manag. Cyber Phys. World (APNOMS)*, 2019, pp. 1–6.

[51] B. Augustin, T. Friedman, and R. Teixeira, "Measuring load-balanced paths in the Internet," in *Proc. 7th ACM SIGCOMM Internet Meas. Conf. (IMC)*, 2007, pp. 149–160.
[52] Y. Li and Y. Lu, "LSTM-BA: DDoS detection approach combining LSTM and Bayes," in *Proc. 7th Int. Conf. Adv. Cloud Big Data (CBD)*, 2019, pp. 180–185.
[53] N. Ravi, S. M. Shalinie, and D. D. J. Theres, "BALANCE: Link flooding attack detection and mitigation via hybrid-SDN," *IEEE Trans. Netw. Service Manag.*, vol. 17, no. 3, pp. 1715–1729, Sep. 2020.
[54] J. Wang, R. Wen, J. Li, F. Yan, B. Zhao, and F. Yu, "Detecting and mitigating target link-flooding attacks using SDN," *IEEE Trans. Depend. Secure Comput.*, vol. 16, no. 6, pp. 944–956, Dec. 2019.
[55] O. Alkadi, N. Moustafa, B. Turnbull, and K.-K. R. Choo, "A deep blockchain framework-enabled collaborative intrusion detection for protecting IoT and cloud networks," *IEEE Internet Things J.*, vol. 8, no. 12, pp. 9463–9472, Jun. 2021.

Xiaofeng Jiang (Member, IEEE) received the B.E. and Ph.D. degrees in information science and technology from the University of Science and Technology of China, Hefei, China, in 2008 and 2013, respectively, where he is currently an Associate Professor with the School of Information Science and Technology. He is also with the Institute of Artificial Intelligence, Hefei Comprehensive National Science Center. His recent research interests include discrete event dynamic system, tensor analysis and big data, future network, and cognitive communications.

Qianbao Shi received the M.S. degree in information science and technology from the University of Science and Technology of China, Hefei, China, in 2013, where she is currently pursuing the Ph.D. degree. She is with the Department of Automation, USTC and the Institute of Artificial Intelligence, Hefei Comprehensive National Science Center, Hefei. Her research interests include aod hoc networks and unmanned aerial vehicle networks.

Hengkun Miao received the B.S. degree in aircraft manufacturing engineering from the Nanjing University of Aeronautics and Astronautics, Nanjing, China, in 2020. He is currently pursuing the M.S. degree in electronic and information engineering with the University of Science and Technology of China, Hefei, China. He is also with the Institute of Artificial Intelligence, Hefei Comprehensive National Science Center. His research interests include edge computing, blockchain, space-air-ground network, computation offloading, and resource allocation.



Wanqin Cao received the M.S. degree in information science and technology from the University of Science and Technology of China, Hefei, China, in 2022. Her research interests include the blockchain technology and networks security.

Huasen He (Member, IEEE) received the B.S. degree in automation from the University of Science and Technology of China (USTC), Hefei, China, in 2013, and the M.S. degree in signal processing and communications and the Ph.D. degree in digital communications from the University of Edinburgh, Edinburgh, U.K., in 2014 and 2018, respectively. He is currently an Associate Research Fellow with the School of Information Science and Technology, USTC. He is also with the Institute of Artificial Intelligence, Hefei Comprehensive National Science Center. His research interests include future networks, network modeling, and optimization.

Shuangwu Chen (Member, IEEE) received the B.S. and Ph.D. degrees from the University of Science and Technology of China, Hefei, China, in 2011 and 2016, respectively, where he is currently an Associate Professor with the School of Information Science and Technology. He is also with the Institute of Artificial Intelligence, Hefei Comprehensive National Science Center. His research interests include multimedia communications, future network, and stochastic optimization.

Jian Yang (Senior Member, IEEE) received the B.S. and Ph.D. degrees from the University of Science and Technology of China, Hefei, China, in 2001 and 2006, respectively, where he is currently a Professor with the School of Information Science and Technology. He is also with the Institute of Artificial Intelligence, Hefei Comprehensive National Science Center. His research interests include future network, distributed system design, modeling and optimization, multimedia over wired and wireless, and stochastic optimization. Dr. Yang received the Lu Jia-Xi Young Talent Award from the Chinese Academy of Sciences in 2009.